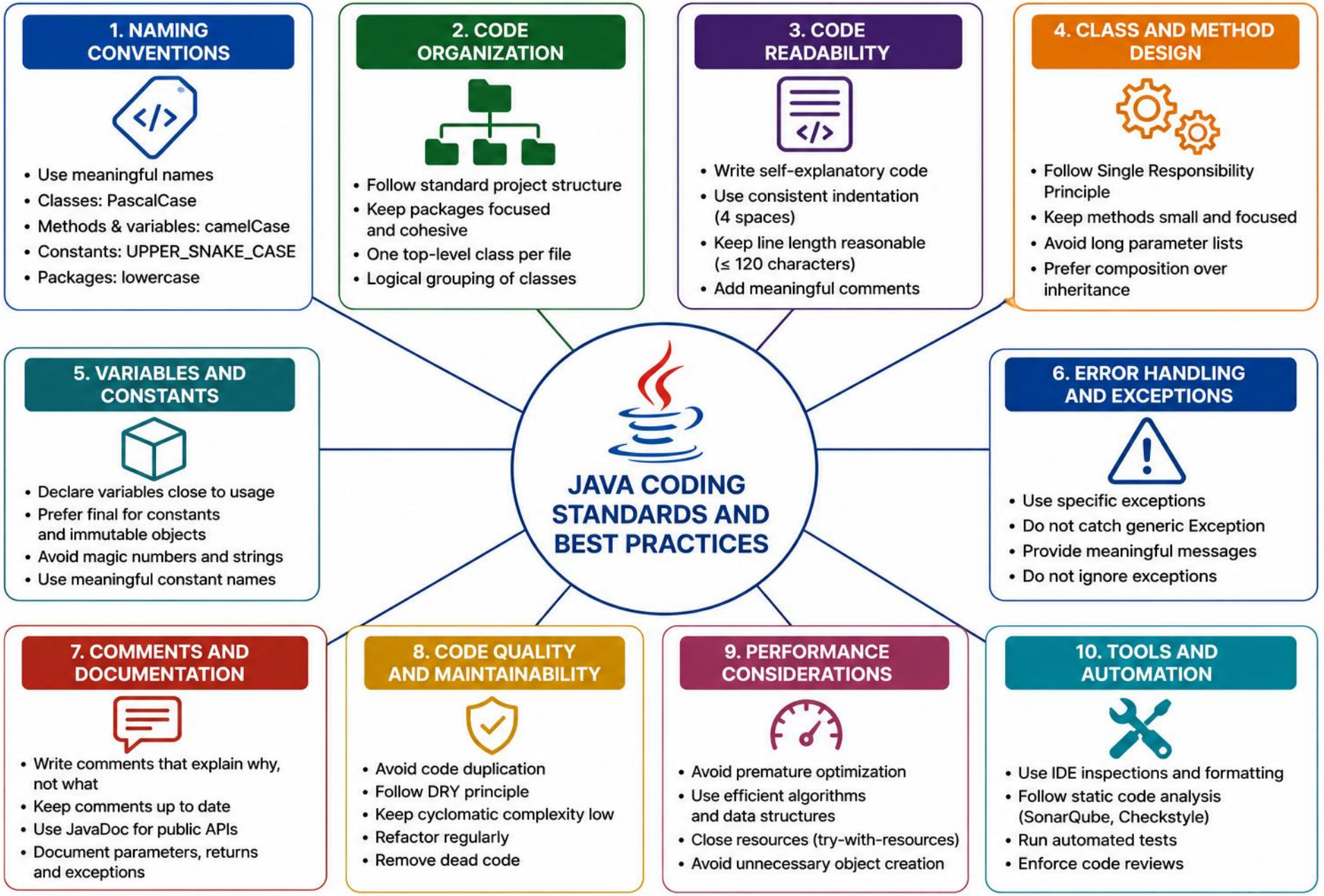
MODULE 12

# Java Coding Standards and Best Practices

Write clean, consistent, maintainable Java code that's easy to read, test, and evolve in enterprise environments.

TEKsystems Global Services

# WORKFORCE DEVELOPMENT



# MODULE 12 OVERVIEW

*Enterprise systems live for years — code must be readable, maintainable, and consistent.*

- Building on Modules 10-11's AI-assisted workflows, this module covers the coding standards and refactoring discipline that keeps a codebase healthy long after AI generates the first draft.

## DURATION & LAB

**2 Hours Theory**

Standards, design principles, refactoring.

**45 Minutes Lab**

Coding Standards and Refactoring.

**Scenario**

Bring a legacy CRM CustomerService up to standard.

## STANDARDS COVER

Naming, formatting, structure, testability, docs

## TOOLS ENFORCE

Checkstyle, PMD, SpotBugs, SonarQube, IDE inspections

## THE PAYOFF

Fewer defects, faster onboarding, lower cost

*Learning objectives: naming/formatting, clean class & method design, exceptions/logging, testable code.*

**KEY TAKEAWAY** Consistent standards + clean code + regular reviews = high-quality, maintainable Java applications.

# WHY CODING STANDARDS MATTER

*Coding standards are a shared set of rules and conventions — the foundation of professional, maintainable software.*

## WITHOUT STANDARDS

- Inconsistent code — hard to read and understand
- Difficult collaboration and onboarding
- Harder maintenance, more defects
- Slower delivery, higher cost

VS

## WITH STANDARDS

- Consistency and predictability across the codebase
- Stronger collaboration and shared understanding
- Better quality, easier long-term maintenance
- Faster delivery, lower risk and cost

## THE BUSINESS IMPACT

**Lower Risk**

**Cost Savings**

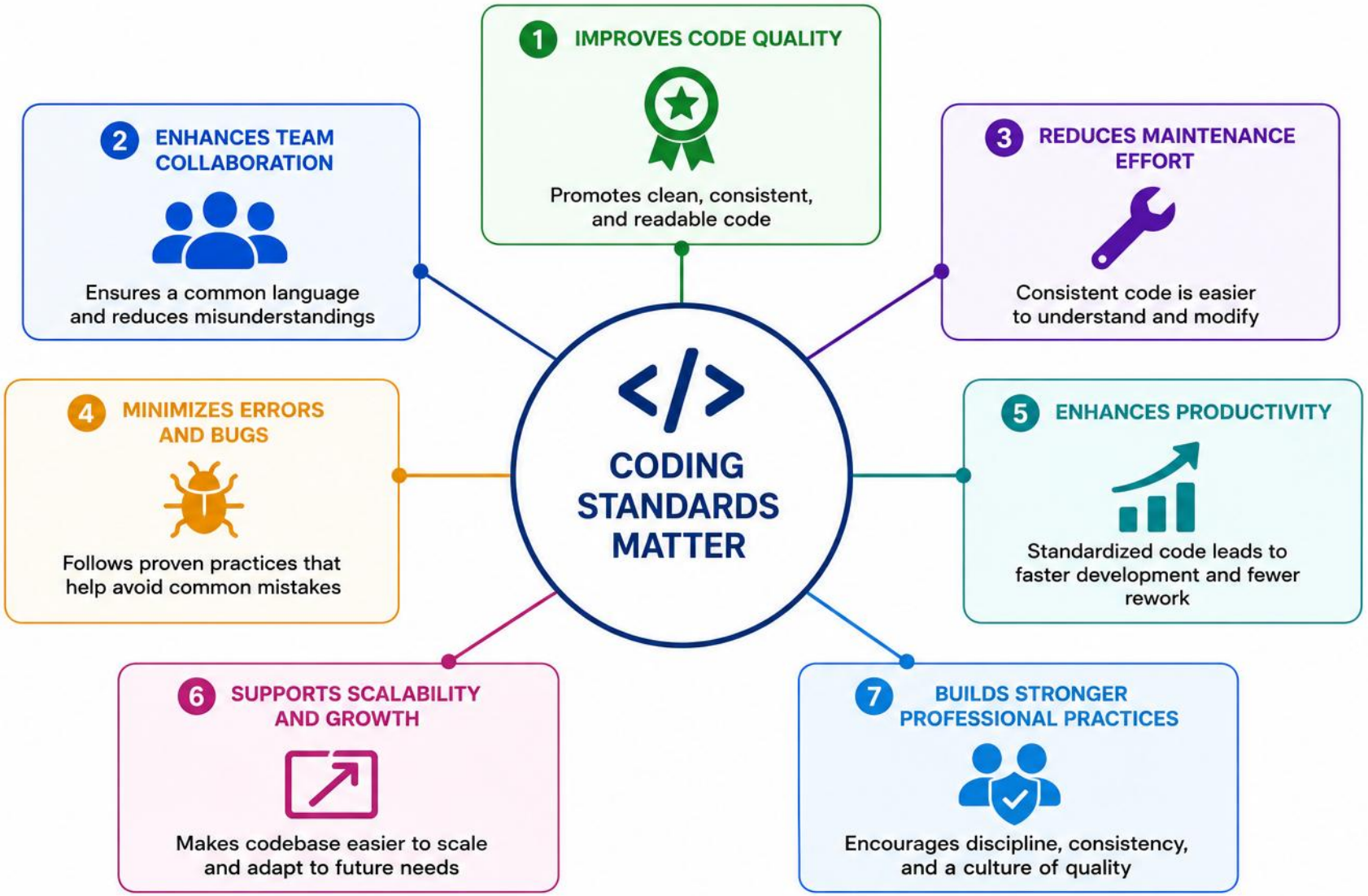
**Faster Time to Market**

**Scalable Growth**

**Professional Excellence**

**KEY TAKEAWAY** Coding standards aren't about restrictions — they enable clarity, consistency, and quality.

# Why Coding Standards Matter



# ENTERPRISE CODING STANDARDS

*Enterprise standards define how we write, structure, and document code across the organization.*

## CORE STANDARDS IN THIS MODULE

### Naming

- Consistent, meaningful identifiers.

### Formatting

- Automated indentation and style.

### Method & Class Design

- Cohesive, well-structured units.

### Readability & Complexity

- Clear code; complexity kept manageable.

### Refactoring

- Safe, incremental improvement.

### Static Analysis & Code Review

- Automated tooling plus team review.

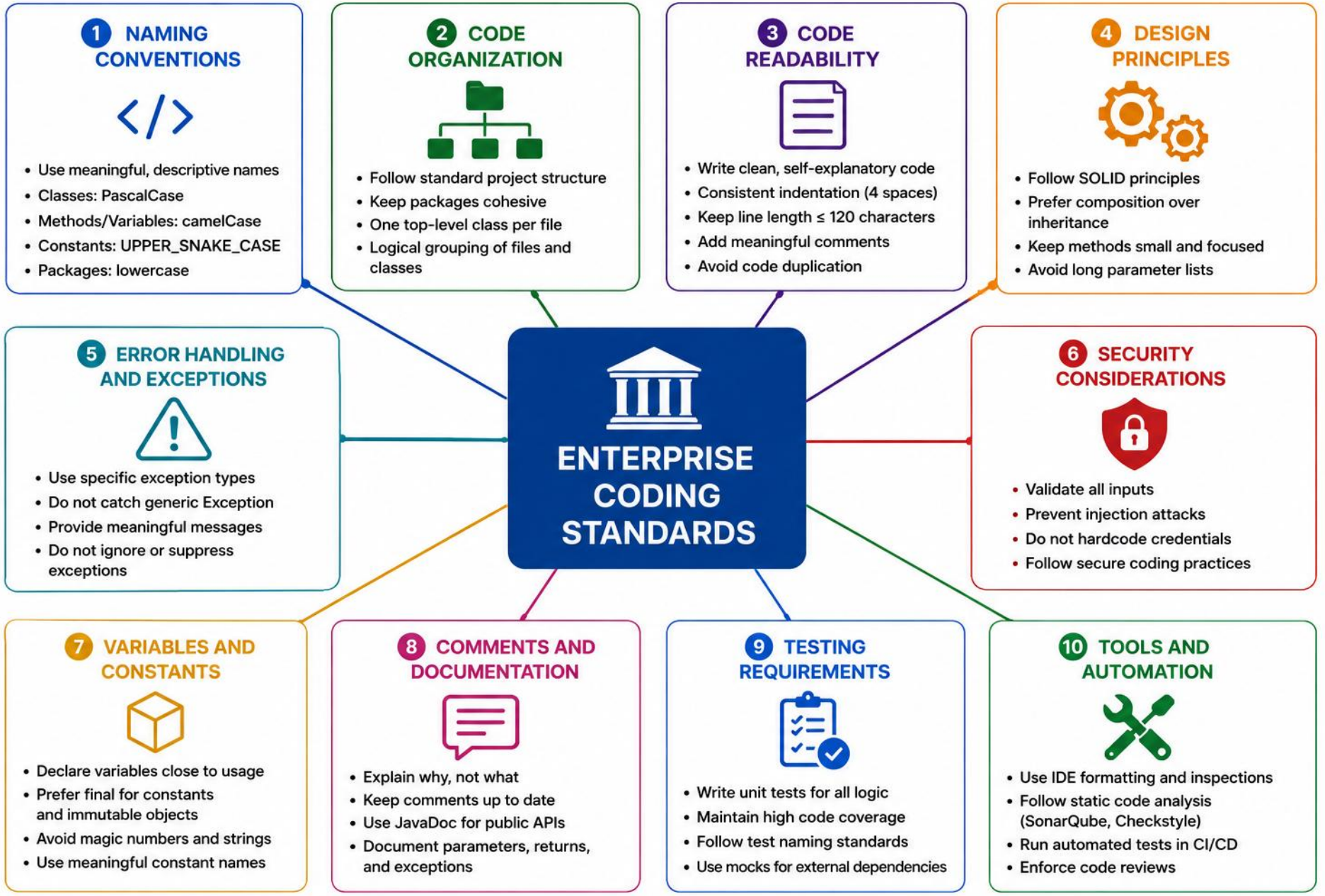
# ENTERPRISE CODING STANDARDS — CROSS-REFERENCES

*Where each standard is covered in greater depth elsewhere in the bootcamp.*

**CROSS-REFERENCES**

- Exceptions — covered in depth in Module 7.
- Structure — covered in depth in Module 8.
- Dependencies — covered in depth in Module 9.
- Security — covered in a later module.

**KEY TAKEAWAY** Enterprise coding standards create a common language, improve quality, reduce risk, and accelerate delivery.



# NAMING CONVENTIONS

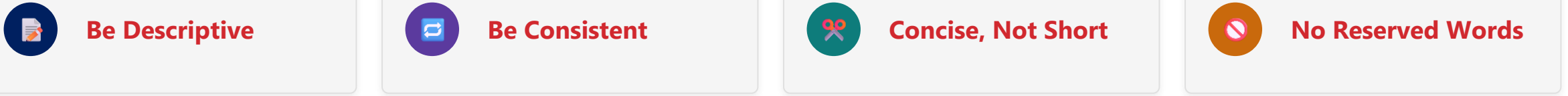
*Use meaningful names. Write self-documenting code — consistent naming makes code easier to read and maintain.*

| ELEMENT | STYLE | RULE | EXAMPLE |
|---|---|---|---|
| `Package` | lowercase | Reverse domain name | com.company.project |
| `Class / Interface` | PascalCase | Noun or noun phrase | OrderService |
| `Method` | camelCase | Verb or verb phrase | calculateTotalAmount() |
| `Variable` | camelCase | Meaningful, no encodings | customerCount |
| `Constant (static final)` | UPPER_SNAKE_CASE | Words separated by _ | MAX_RETRY_COUNT |
| `Enum type` | PascalCase | Noun, singular | PaymentStatus |
| `Enum constant` | UPPER_SNAKE_CASE | Words separated by _ | PAYMENT_COMPLETED |
| `Type Parameter` | Single letter | Letter, or descriptive if unclear | <T>, <K, V>, <TEntity> |
| `Field (instance)` | camelCase | Noun or noun phrase | orderRepository |

# NAMING CONVENTIONS — BEST PRACTICES

*Four habits that keep names consistent, readable, and easy to search across the codebase.*

**BEST PRACTICES**




**KEY TAKEAWAY** Good names are the foundation of clean code — follow conventions, be consistent, make your code self-documenting.

# Naming Conventions

## 1 CLASSES

- Use PascalCase
- Noun or noun phrase
- Singular

**Example:**
**CustomerService**

## 2 INTERFACES

- Use PascalCase
- Adjective or noun phrase
- Often start with an "I"

**Example:**
**IPaymentProcessor**

## 3 METHODS

- Use camelCase
- Verb or verb phrase
- Start with lowercase letter

**Example:**
**calculateTotal()**

## 4 VARIABLES

- Use camelCase
- Noun or noun phrase
- Start with lowercase letter

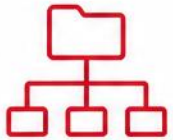
**Example:**
**customerName**

## 5 CONSTANTS

- Use UPPER_SNAKE_CASE
- Noun or noun phrase
- Use meaningful names

**Example:**
**MAX_RETRY_COUNT**

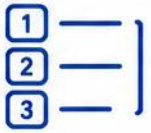
## 6 PACKAGES



- Use lowercase
- Use nouns
- Separate words with dots

**Example:**
**com.company.project.service**

## 7 ENUMS




- Use PascalCase
- Singular noun
- Group related constants

**Example:**
**OrderStatus**

## 8 TYPE PARAMETERS


- Use single uppercase letter
- Use meaningful letters
- Common: T, E, K, V

**Example:**
**Map<K, V>**

# NAMING CONVENTIONS IN PRACTICE

*The same naming rules from the table, applied to one small class.*

**BEFORE / AFTER EXAMPLE**

```
// Bad — unclear names, wrong casing
class custSvc {
    static final int max = 3;
    String CustName;
}

// Good — meaningful names, correct casing
public class CustomerService {
    private static final int MAX_RETRY_COUNT = 3;
    private String customerName;
}
```

**KEY TAKEAWAY** Good names say what something is or does — casing carries meaning, not decoration.

# METHOD DESIGN PRINCIPLES AND CHECKLIST

*Well-designed methods make code easy to read, test, and maintain — with a quick checklist and naming examples to apply them.*

### Single Responsibility

- A method should do one thing and do it well.

### Short and Focused

- Easy to follow at one level of abstraction — length is a signal, not a rule.

### Clear Parameters

- Related values traveling together? Consider a parameter object.

### Avoid Side Effects

- Don't modify external state unexpectedly.

### Return Meaningful Values

- Return a value when callers need one; void suits clear commands (e.g., sendNotification()).

### Handle Exceptions

- Don't swallow exceptions; add context.

# METHOD DESIGN — ANTI-PATTERNS TO AVOID

*Naming and structure smells to watch for, with the checklist you just covered.*

**ANTI-PATTERNS TO AVOID**

- Long methods mixing responsibilities; parameter lists that are hard to follow; generic names like process(), handle(), doStuff().
- Good names describe purpose: calculateInvoiceTotal(), isEligibleForDiscount(), validateEmailAddress().
- Avoid vague names: process(), run(), handleData(), temp(), doStuff().
- Checklist: name matches purpose; minimal meaningful parameters; no duplication; guard clauses; easy to test in isolation.

**KEY TAKEAWAY** If you can't explain a method's purpose in one sentence, it's doing too much.

# METHOD DESIGN: BEFORE / AFTER

*One responsibility, a clear name, and a guard clause — the checklist applied.*

**BEFORE / AFTER EXAMPLE**

```
// Bad — vague name, mixes math and side effects
void process(Order o) {
    double t = 0;
    for (Item i : o.getItems()) t += i.getPrice();
    o.setTotal(t);
}

// Good — single responsibility, clear name, guard clause
double calculateOrderTotal(Order order) {
    if (order == null) throw new IllegalArgumentException("order");
    return order.getItems().stream()
        .mapToDouble(Item::getPrice).sum();
}
```

**KEY TAKEAWAY** If you can't explain a method in one sentence, split it — do exactly one thing.

# TRY IT YOURSELF — EXERCISE 1: TARGET API SKETCH

*Reinforces: designing a clean method signature before refactoring — the checklist you just covered.*

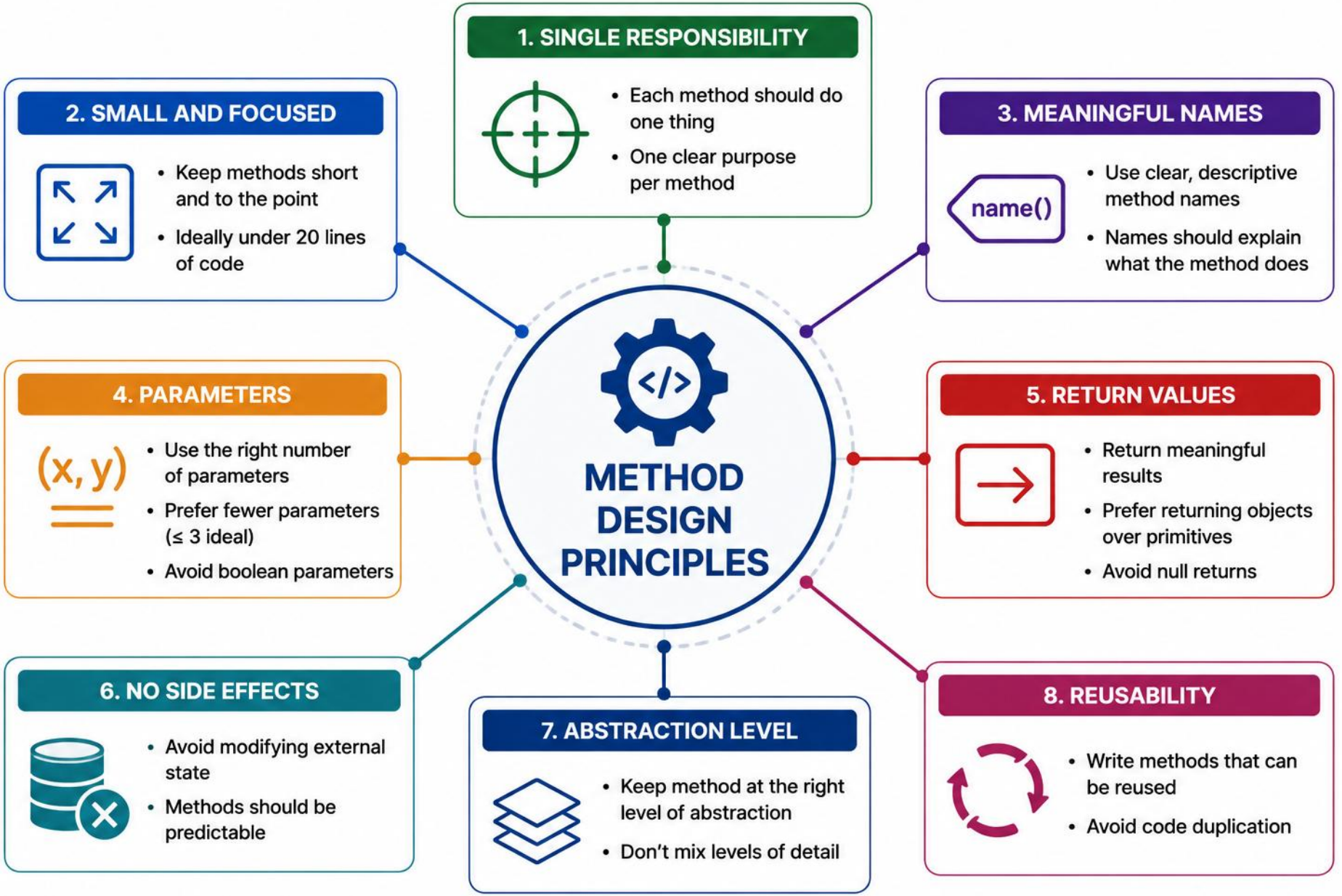
| STEP | WHAT TO DO |
|---|---|
| **Goal** | Sketch a minimal CustomerService API (paper only) before touching the legacy refactor |
| **File** | target-api-sketch.md (in examples/module-12-exercises/notes/) |
| **Methods** | findById, activateProspect, maybe validateStatus — names and intent only, no bodies |
| **Ravi path** | activateProspect(CUS-1002) moves Ravi's status PROSPECT -> ACTIVE |
| **Keep out** | No SOAP endpoints, no Spring controllers — sketch only, not the full Lab 12 refactor |
| **Reference** | exercises/exercise-01-target-api-sketch.md |

# TRY IT YOURSELF — EXERCISE 1: TARGET API SKETCH

*Reference code — compare your answer, then continue.*

```
$ cat target-api-sketch.md
  CustomerService.findById(String customerId)
  CustomerService.activateProspect(String customerId)  // CUS-1002 Ravi: PROSPECT -> ACTIVE
  CustomerService.validateStatus(String status)     // maybe -- confirm before adding
  // excluded: SOAP endpoints, @Controller, @RestController
```

**TRY IT NOW** Complete Exercise 1 before continuing — see exercises/exercise-01-target-api-sketch.md.



# CLASS DESIGN PRINCIPLES: SOLID IN PRACTICE

*Well-designed classes are the foundation of clean, maintainable software — with anti-patterns to avoid and quick tips for applying SOLID.*

### High Cohesion

- A class should represent one cohesive responsibility — not mix unrelated business, infrastructure, and presentation concerns.

### Encapsulation

- Hide internal state with private fields.

### Abstraction

- Expose only essential details through public methods.

### Open/Closed Principle

- Design stable extension points where variation is expected; refactor existing code when that's simpler and clearer.

### Favor Composition

- Use composition over inheritance for flexibility.

### Keep Classes Small

- Small classes are easier to understand and test.

# CLASS DESIGN — SOLID SUMMARY & ANTI-PATTERNS

*The five principles in one line, plus the anti-patterns to avoid.*

**SOLID PRINCIPLES**

- Single Responsibility • Open/Closed • Liskov Substitution • Interface Segregation • Dependency Inversion.
- ANTI-PATTERNS TO AVOID — God classes with too many responsibilities; public fields exposing internal state; inheritance used where composition fits better; deep hierarchies; unclear class names.

**KEY TAKEAWAY** If the class name is hard to explain, rethink the design — encapsulate data and keep classes focused.

# SOLID IN PRACTICE

*Single Responsibility Principle: split two reasons to change into two classes.*

**BEFORE / AFTER EXAMPLE**

```
// Bad — one class owns both business rules and persistence
class InvoiceService {
    double calculateTotal(Invoice inv) { return 0; }
    void saveToDatabase(Invoice inv) { }
}

// Good — one reason to change per class
class InvoiceCalculator {
    double calculateTotal(Invoice inv) { return 0; }
}
class InvoiceRepository {
    void save(Invoice inv) { }
}
```

**KEY TAKEAWAY** One reason to change per class — split responsibilities instead of stacking them.

# TRY IT YOURSELF — EXERCISE 2: SOLID APPLY VS. DEFER

*Reinforces: the SOLID principles you just covered — deciding what to apply now vs. defer.*

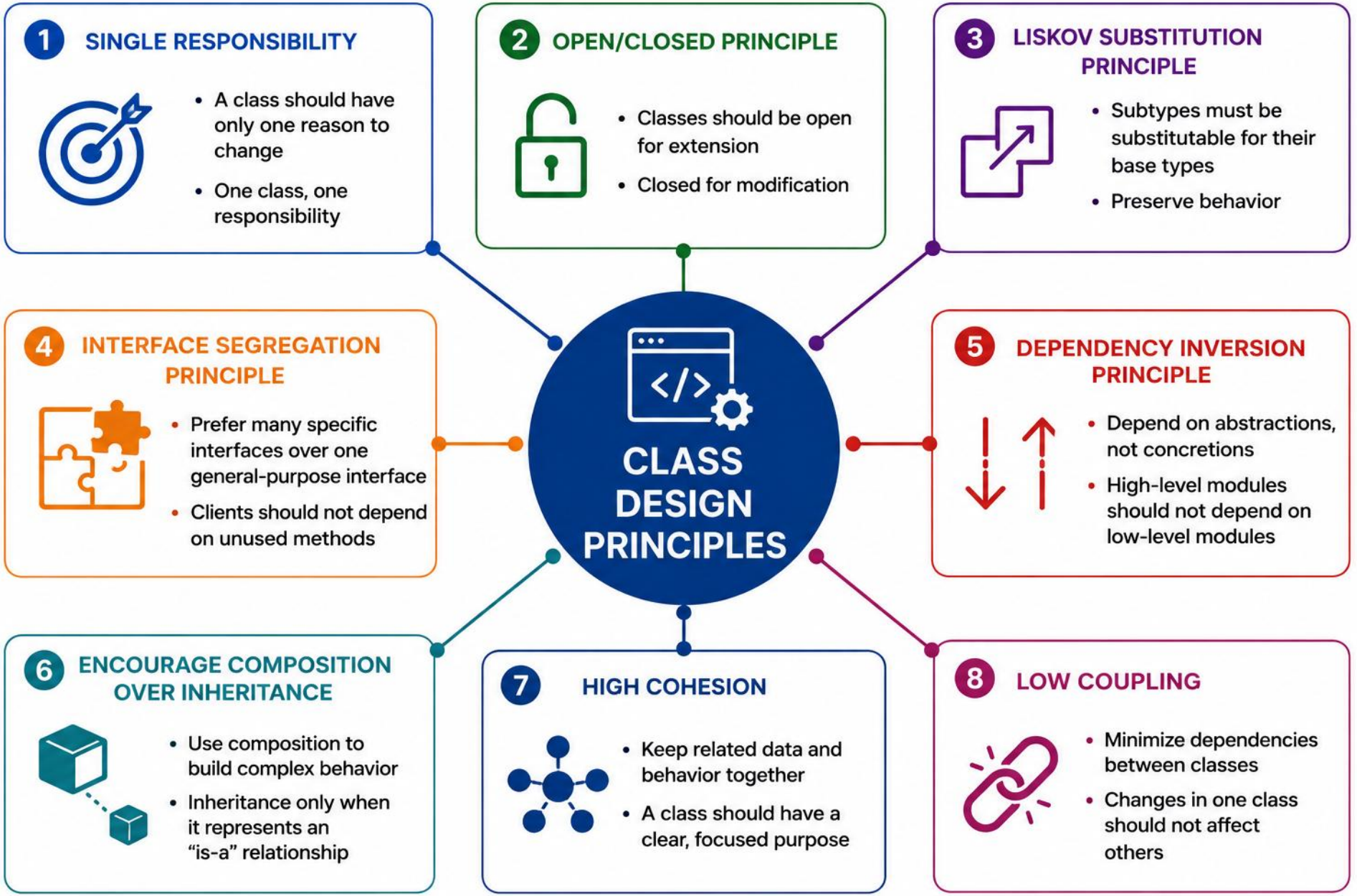
| STEP | WHAT TO DO |
|---|---|
| **Goal** | Choose one SOLID item to apply now and two to defer before the SOAP/Spring labs |
| **File** | lab12-solid-scope.md (in examples/module-12-exercises/notes/) |
| **Apply now** | SRP — separate a validation helper from persistence-shaped code in the sketch |
| **Defer** | DIP wiring frameworks and ISP for large SOAP ports — defer to Labs 13+ |
| **Why defer** | One sentence: Modules 10-12 stay before SOAP; do not over-architect ports yet |
| **Reference** | exercises/exercise-02-solid-apply-defer.md |

# TRY IT YOURSELF — EXERCISE 2: SOLID APPLY VS. DEFER

*Reference code — compare your answer, then continue.*

```
$ cat lab12-solid-scope.md
  Apply now: SRP -- separate validation helper from persistence-shaped code
  Defer: DIP (framework wiring), ISP (large SOAP ports)
    - revisit in Labs 13+ once SOAP/Spring are introduced
  // do not over-architect ports before they exist
```

**TRY IT NOW** Complete Exercise 2 before continuing — see exercises/exercise-02-solid-apply-defer.md.



# BEFORE / AFTER: READABILITY REFACTOR

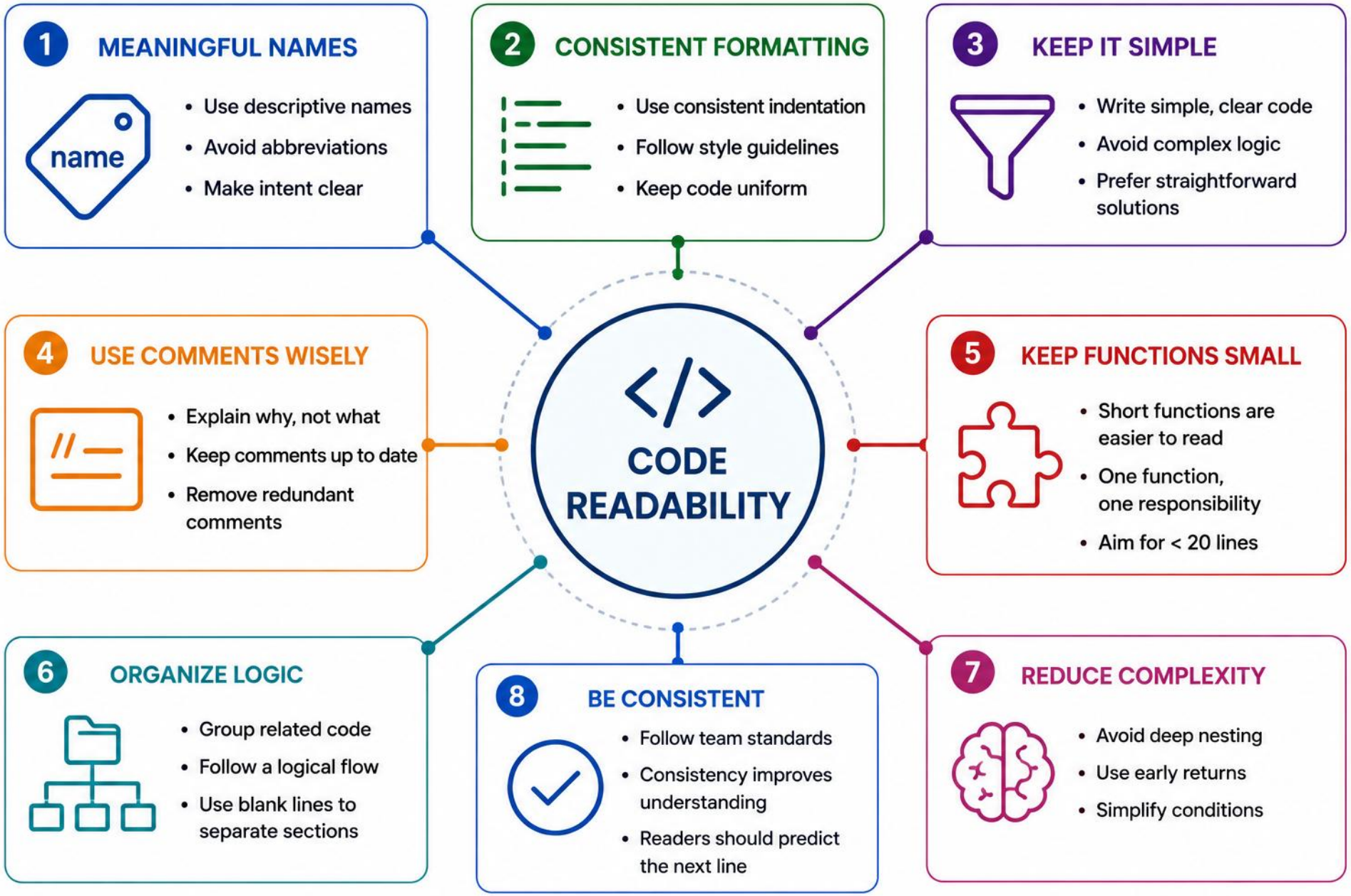
*Readable code is easier to understand, review, test, and extend — see the difference a small refactor makes.*

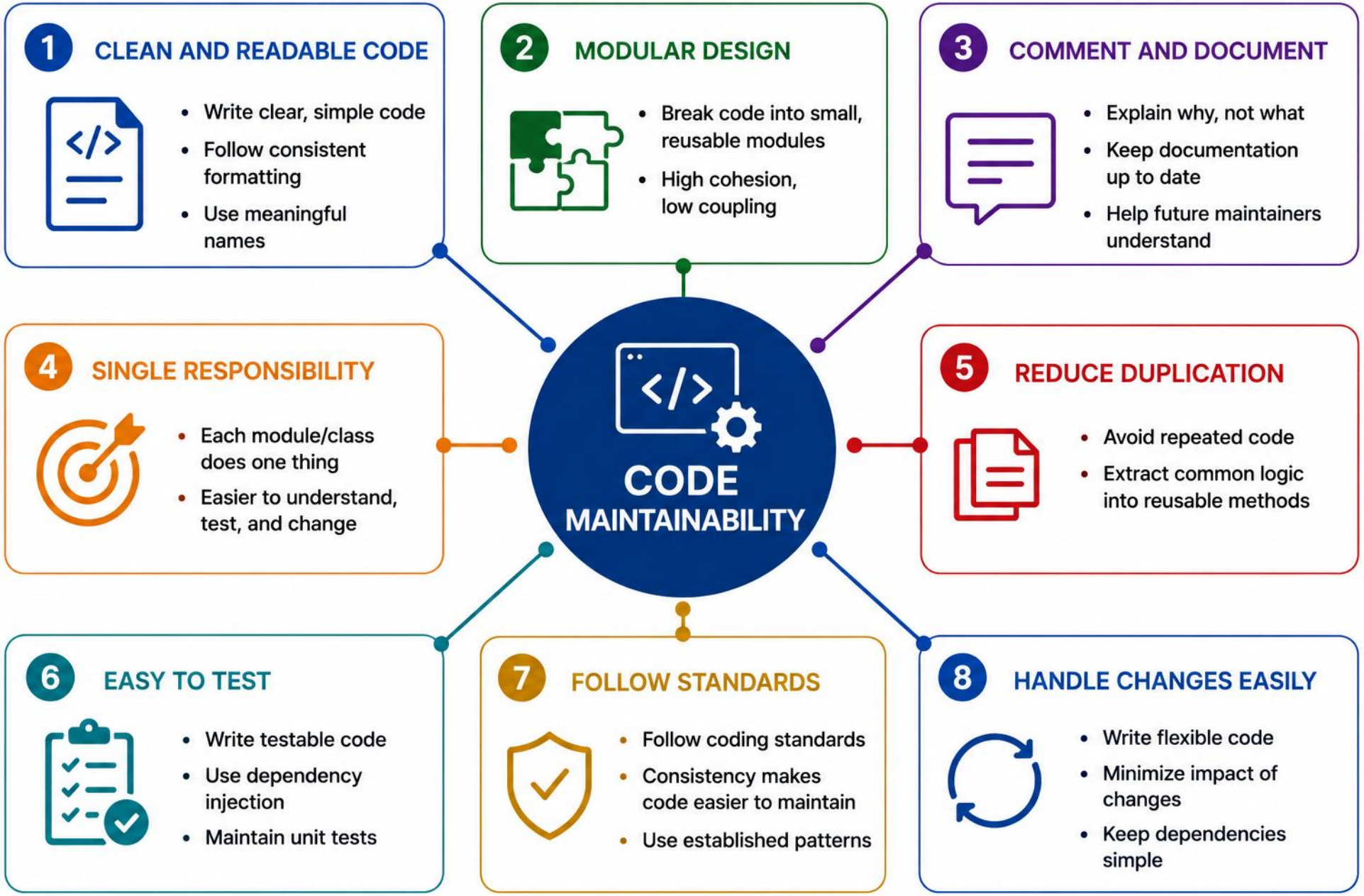
**BEFORE / AFTER EXAMPLE**

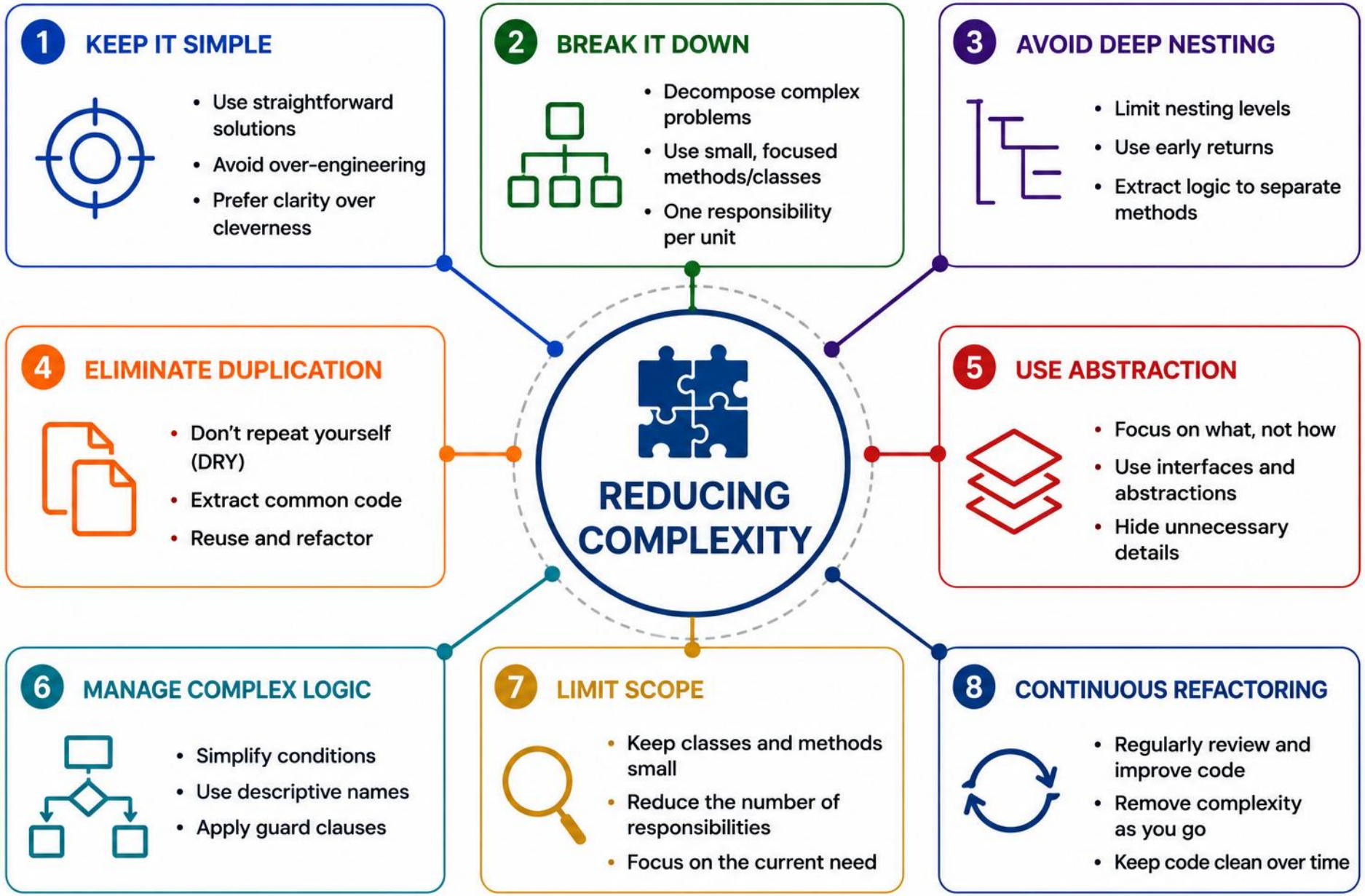
```
// Bad
int d; String n;

// Good
int daysSinceLastLogin;
String customerName;
```

- Quick tips: read your code aloud, use IDE formatting tools, refactor early and often.
- Tools that help: Checkstyle, PMD, SonarQube, IntelliJ IDEA.

**KEY TAKEAWAY** Clean, well-structured code today becomes easy to maintain, extend, and scale tomorrow.







# COMMON CODE SMELLS (1 OF 2)

*Code smells don't always mean the code is wrong — they signal something may be harder to maintain or change.*

| CODE SMELL | WHAT IT LOOKS LIKE | HOW TO IMPROVE |
|---|---|---|
| `Long Method` | A method is very long, doing many things | Extract Method — break into small, focused methods |
| `Duplicated Code` | Same code appears in multiple places | Extract Method — create one method and reuse it |
| `Long Parameter List` | Method requires many parameters (4+) | Introduce Parameter Object |
| `Deep Nesting` | Multiple levels of if/else or loops | Guard Clauses — return early, reduce nesting |
| `Magic Numbers/Strings` | Hard-coded literals scattered in code | Use Constants/Enums |

# COMMON CODE SMELLS (2 OF 2)

*More smells to recognize — and how to fix each one.*

| CODE SMELL | WHAT IT LOOKS LIKE | HOW TO IMPROVE |
|---|---|---|
| `Shotgun Surgery` | One small change requires edits across many classes | Centralize the rule in one class (e.g., DiscountPolicy) |
| `God Class / Feature Envy` | A class does too much, or a method overly depends on another class's data | Apply Single Responsibility; move logic to the class that owns the data |
| `Data Clumps` | The same group of variables or parameters travels together repeatedly | Introduce Parameter Object or a small value class |
| `Primitive Obsession` | Overusing primitives instead of small domain types | Introduce a domain type (e.g., Money, EmailAddress) |
| `Divergent Change` | One class changes for many unrelated reasons | Split the class by responsibility (apply SRP) |

**KEY TAKEAWAY** Code smells are not bugs — they're early warnings. Fixing them early keeps your code clean and future-ready.

# CODE SMELLS: BEFORE / AFTER

*Magic numbers and a long method, cleaned up with the techniques from this table.*

**BEFORE / AFTER EXAMPLE**

```
// Bad — magic number, long method, unclear boolean
boolean check(Customer c) {
    if (c.getOrders().size() > 5) {
        return true;
    }
    return false;
}

// Good — named constant, guard clause, clear name
private static final int LOYALTY_THRESHOLD = 5;
boolean isLoyaltyCustomer(Customer customer) {
    return customer.getOrders().size() > LOYALTY_THRESHOLD;
}
```

**KEY TAKEAWAY** Extract Method and Replace Magic Number turn a smell into self-documenting code.

# TRY IT YOURSELF — EXERCISE 3: SMELL BINGO

*Reinforces: the code smells table you just covered — spotted in a rushed Customer service sketch.*

| STEP | WHAT TO DO |
|---|---|
| **Goal** | Mark five smells you expect in a rushed Customer service sketch |
| **File** | lab12-smell-bingo.md (in examples/module-12-exercises/notes/) |
| **Bingo card** | Long method, magic strings for ACTIVE/PROSPECT, == on Strings, mixed I/O, unclear names |
| **Fixture tie-in** | For each smell, note how it could corrupt CUS-1001 / CUS-1002 handling |
| **Priority** | Star the two smells you will fix first in the timed lab |
| **Reference** | exercises/exercise-03-smell-bingo.md |

# TRY IT YOURSELF — EXERCISE 3: SMELL BINGO

*Reference code — compare your answer, then continue.*

```
$ cat lab12-smell-bingo.md
  Smell: magic strings (ACTIVE / PROSPECT)   // fix first
  Smell: == on Strings (status, customer id)   // fix first
  Smell: long method, mixed I/O in domain
  Smell: unclear names   // note CUS-1001/CUS-1002 impact
```

**TRY IT NOW** Complete Exercise 3 before continuing — see exercises/exercise-03-smell-bingo.md.

# TRY IT YOURSELF — EXERCISE 4: EQUALS VS. ==

*Reinforces: the magic-strings smell you just saw — safe comparisons for status and id.*

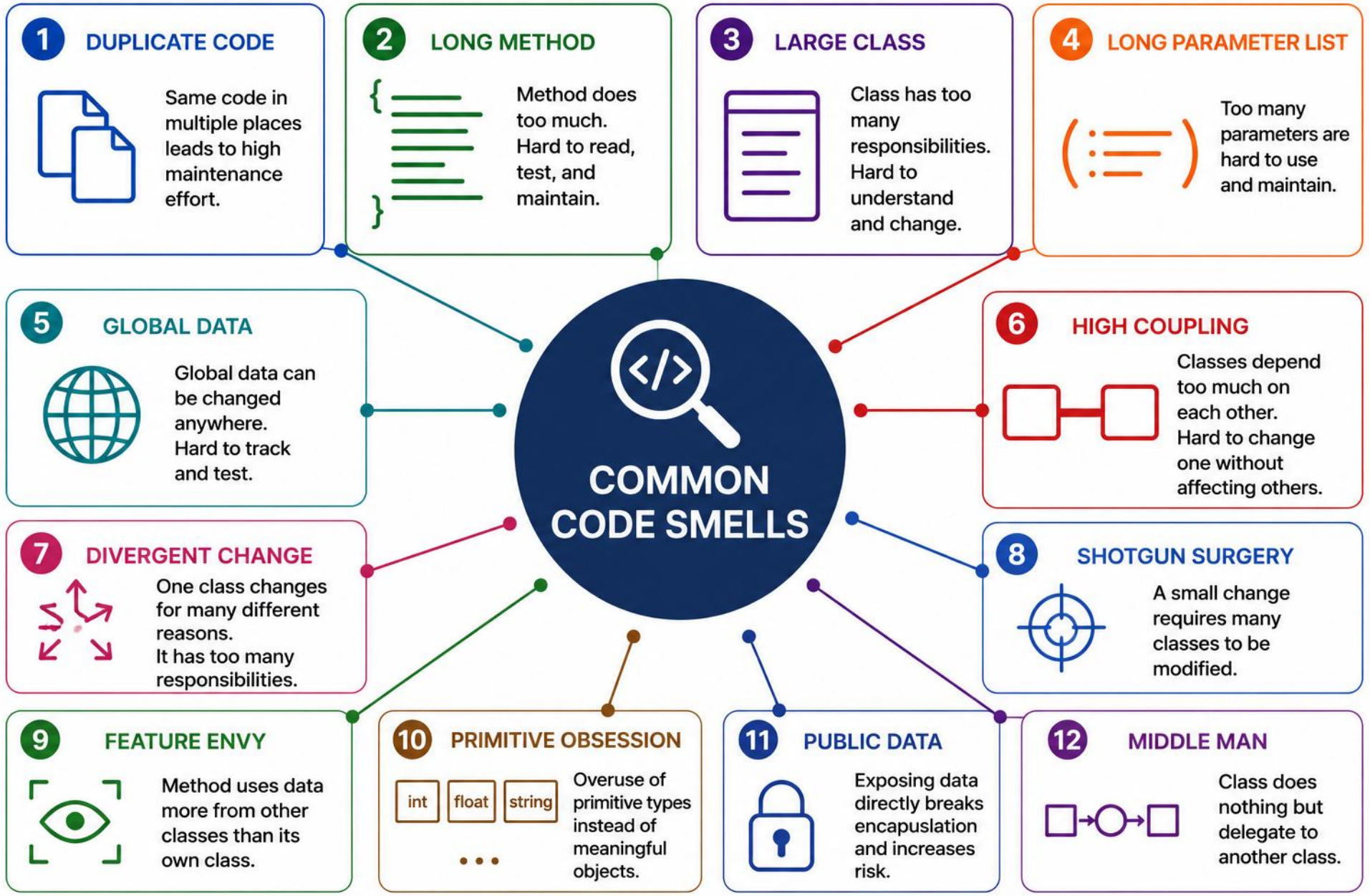
| STEP | WHAT TO DO |
|---|---|
| **Goal** | Document when == is wrong for status strings and customer ids |
| **File** | equals-vs-eqeq.md (in examples/module-12-exercises/notes/) |
| **Reference table** | status ACTIVE? use Objects.equals/enum; same instance? use ==; id CUS-1001? use equals |
| **Bad snippet** | if (status == "ACTIVE") — label it Fail; String identity is unsafe |
| **Good snippet** | Amina ACTIVE checked with equals or enum, plus a null-safe status row you add |
| **Reference** | exercises/exercise-04-equals-vs-eqeq.md |

# TRY IT YOURSELF — EXERCISE 4: EQUALS VS. ==

*Reference code — compare your answer, then continue.*

```
$ cat equals-vs-eqeq.md
  // Bad -- Fail
  if (status == "ACTIVE") { ... }
  // Good -- Amina, status ACTIVE
  if (Objects.equals(status, "ACTIVE")) { ... }   // JDK 21: prefer enum if closed set
```

**TRY IT NOW** Complete Exercise 4 before continuing — see exercises/exercise-04-equals-vs-eqeq.md.



# REFACTORING TECHNIQUES

*Detect complexity and code smells, then refactor safely — a 3-step process for cleaner, easier-to-change code.*

| TECHNIQUE | WHAT IT DOES | BENEFIT |
|---|---|---|
| `Extract Method` | Break a long method into smaller, well-named methods | Improves readability and reusability |
| `Extract Class` | Move related fields/methods to a new class | Better cohesion and separation |
| `Rename` | Use meaningful names for variables, methods, classes | Easier to understand |
| `Introduce Guard Clauses` | Handle edge cases early, reduce nesting | Reduces complexity |
| `Replace Conditional with Polymorphism` | Replace type-checks with polymorphic behavior | Open for extension |

# REFACTORING — THE SAFE PROCESS

*A 3-step process for applying the techniques you just saw — safely.*

**SAFE REFACTORING PROCESS**

1 **Detect the Smell** → 2 **Choose the Refactoring** → 3 **Apply Safely**

- Detect complexity signals (deep nesting, high cyclomatic complexity, mixed abstraction levels); use polymorphism only for stable type variation. Before refactoring, establish a safety net -- tests where possible, or static analysis, version control, and small steps.

**KEY TAKEAWAY** Small, safe changes lead to big, long-term benefit -- always refactor behind a safety net, whether tests, analysis, or small incremental steps.

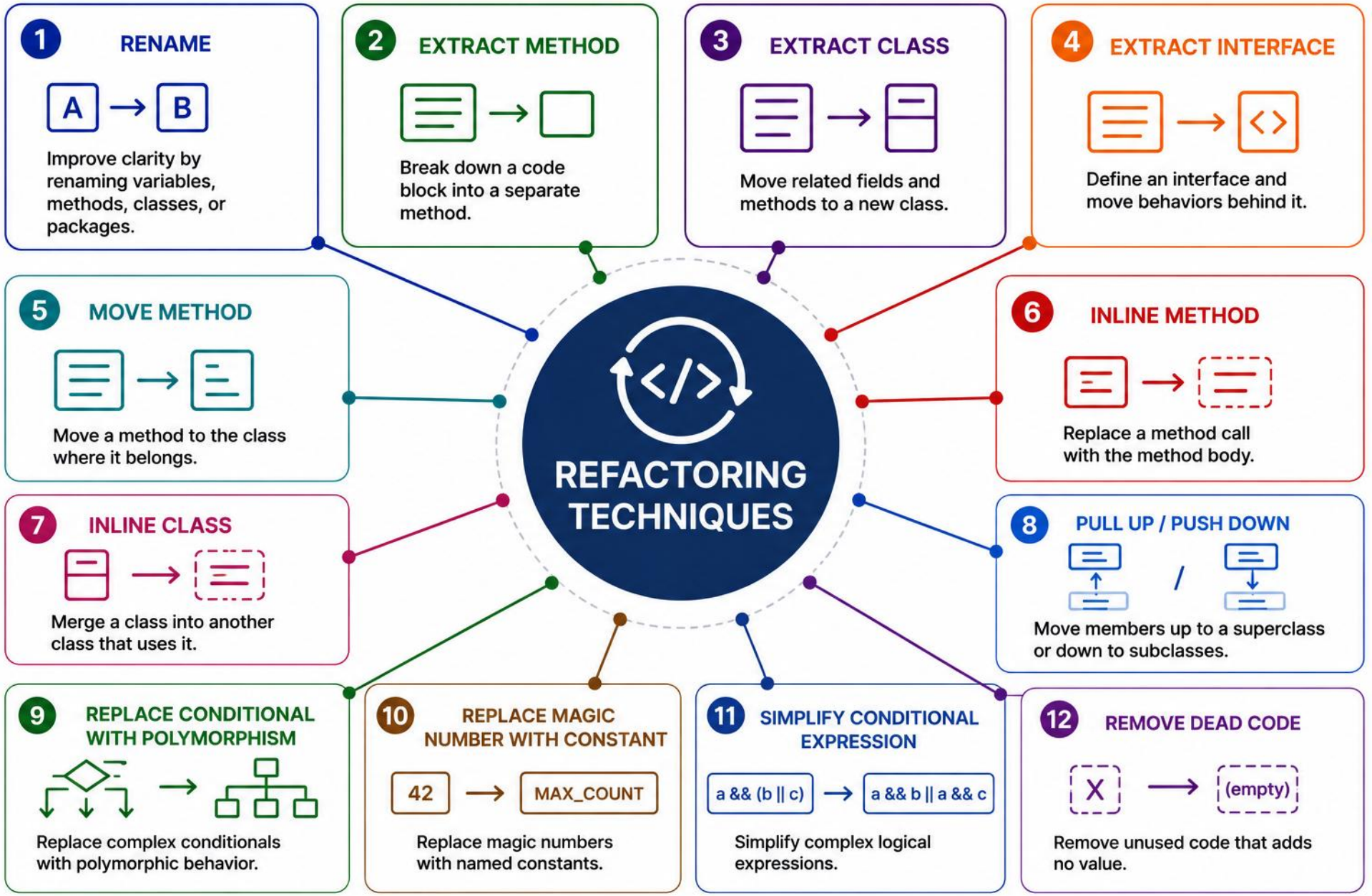
# EXTRACT METHOD IN PRACTICE

*Detect the smell, apply Extract Method, verify behavior with a test — safely.*

**BEFORE / AFTER EXAMPLE**

```
// Bad — one long method, mixed levels of abstraction
void printReport(List<Order> orders) {
    double sum = 0;
    for (Order o : orders) sum += o.getTotal();
    System.out.println("Total: " + sum);
}

// Good — Extract Method separates calculation from output
void printReport(List<Order> orders) {
    System.out.println("Total: " + totalOf(orders));
}
double totalOf(List<Order> orders) {
    return orders.stream().mapToDouble(Order::getTotal).sum();
}
```

**KEY TAKEAWAY** Extract Method turns one tangled block into two small, testable, well-named pieces.



# STATIC ANALYSIS ECOSYSTEM

*Automated tools and CI/CD quality gates that analyze the whole codebase — separate from IntelliJ's live, in-editor feedback.*

### WHAT IT FINDS

- Bugs — e.g. null pointer dereference, resource leaks.
- Security vulnerabilities — e.g. SQL injection.
- Code smells — long methods, duplication.
- Coding standard violations and style issues.

### POPULAR TOOLS (JAVA)

| TOOL | PURPOSE |
|---|---|
| `SonarQube` | Centralized code-quality and security analysis across projects |
| `Checkstyle` | Source-style and convention checks |
| `PMD` | Source-based rules for potential defects and maintainability |
| `SpotBugs` | Bytecode analysis for likely defects |

# STATIC ANALYSIS — BEST PRACTICES

*Habits that make automated analysis actually pay off.*

**BEST PRACTICES**

### Start Early

Integrate SCA in IDE and pipeline.

### Review Reports

Act on issues, not just generate them.

### Measure Progress

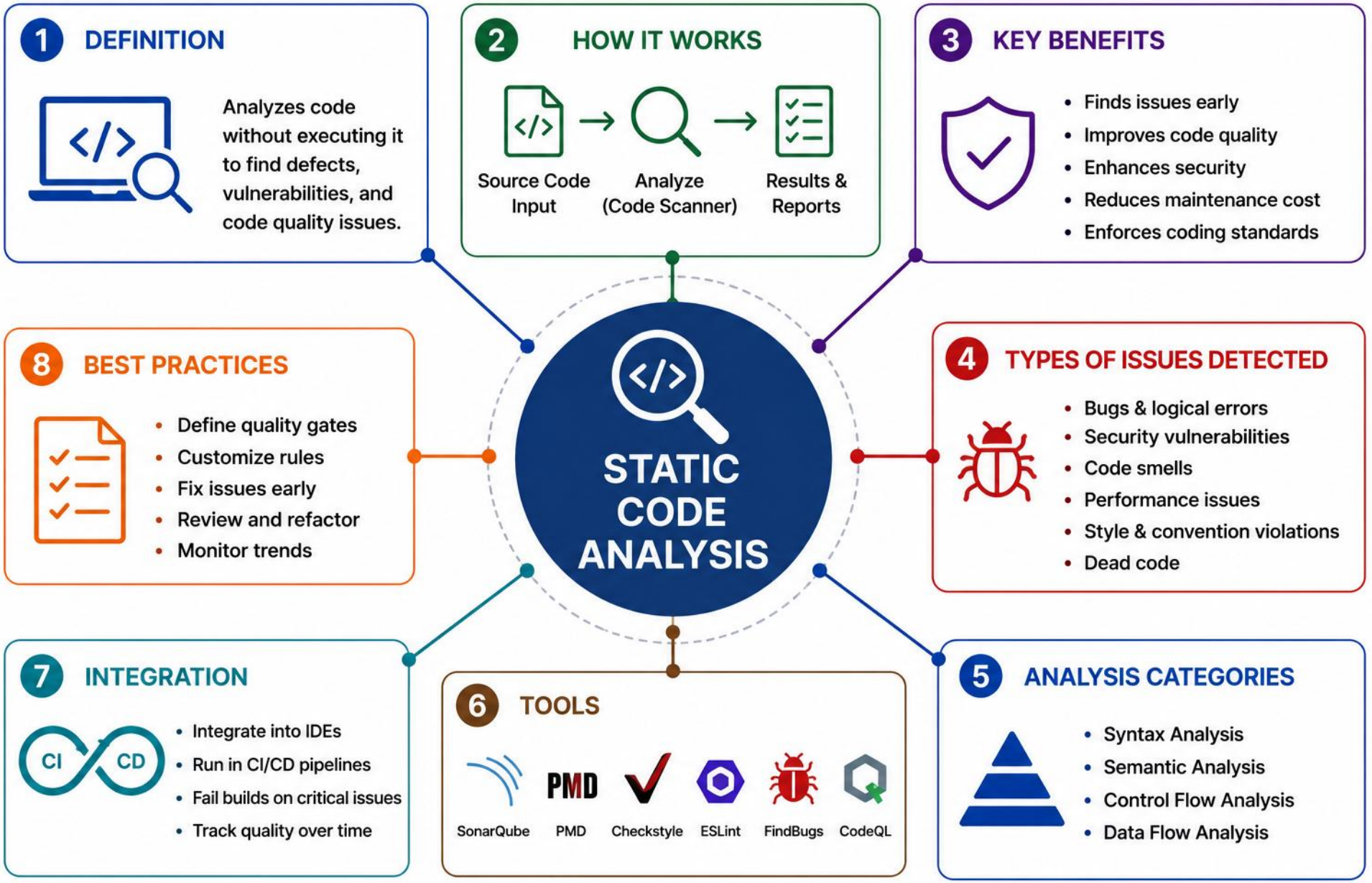
Track and reduce issues over time.

### Focus on Real Impact

Not every warning is a bug.

*Also part of SCA: syntax, semantic, control-flow & data-flow analysis, wired into CI/CD quality gates that fail builds on violations.*

**KEY TAKEAWAY** Analyze early, fix early -- automated tools plus CI/CD gates catch what manual review misses.



# INTELLIJ CODE INSPECTIONS

*IntelliJ IDEA gives local, in-editor feedback as you type -- quick fixes and a team-shared inspection profile, distinct from CI/CD-wide static analysis.*

## TYPES OF INSPECTIONS

### Probable Bugs

Detects likely bugs and runtime issues.

### Code Smells

Maintainability and readability problems.

### Security Vulnerabilities

Insecure APIs, weak crypto, injection risks.

### Formatting & Style

Enforces code style and formatting rules.

### Performance Issues

Inefficient code and resource usage.

### Duplicate Code

Copy-pasted or repeated logic.

# INTELLIJ INSPECTIONS — SHORTCUTS & SEVERITY

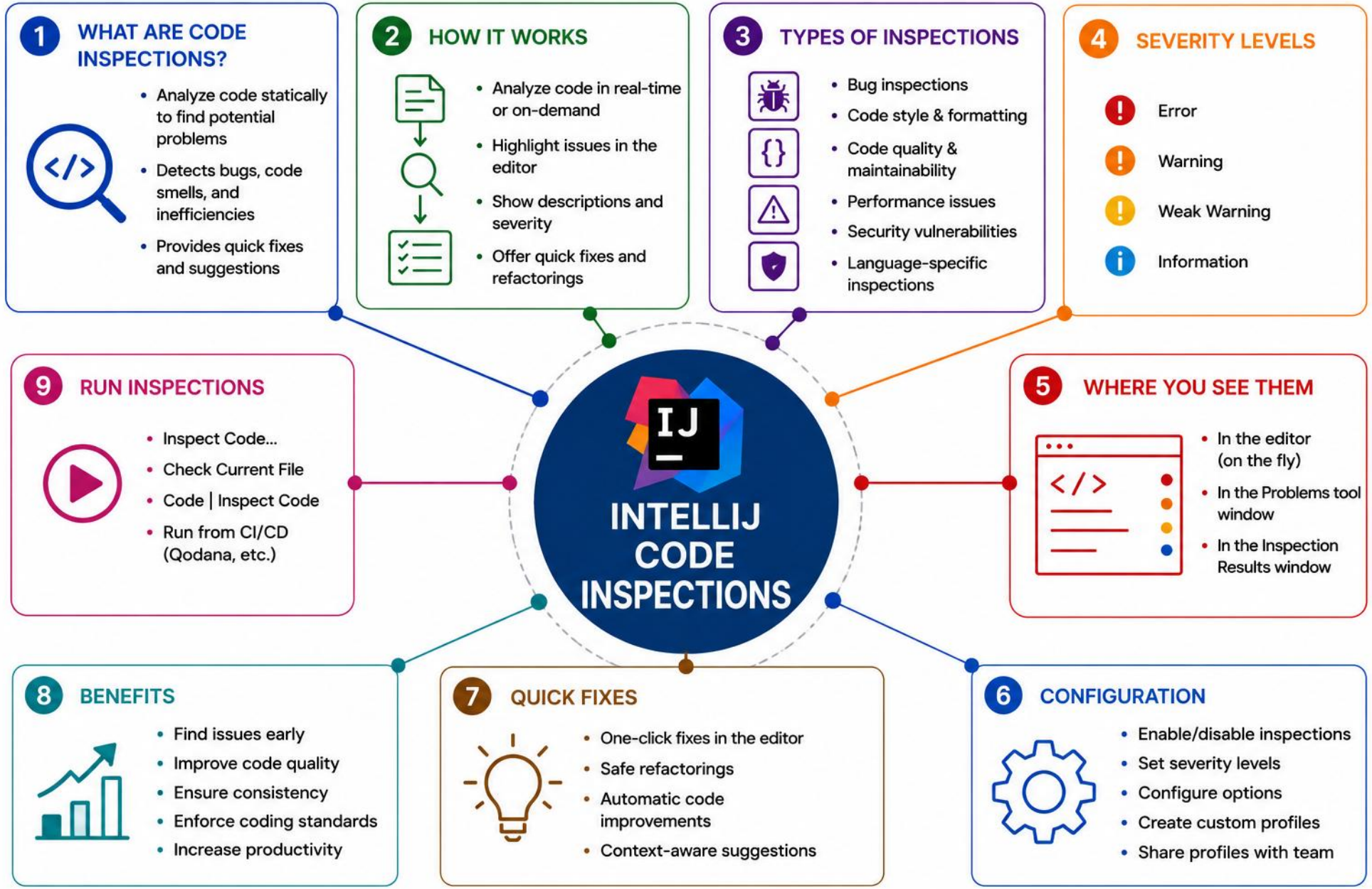
*Keyboard shortcuts to work faster, and how severity levels are configured.*

## USEFUL SHORTCUTS (WINDOWS/LINUX DEFAULTS)

- Inspect Code: Ctrl+Alt+Shift+I • Quick Fix: Alt+Enter • Reformat Code: Ctrl+Alt+L • Configure Inspections: Ctrl+Alt+S
- Shortcuts vary by operating system and keymap -- use Find Action (press Shift twice) to locate any command by name.

*Severity levels: Error > Warning > Weak Warning > Info — configurable per rule under Settings → Editor → Inspections.*

**KEY TAKEAWAY** Fix issues as IntelliJ flags them -- catching problems while you type is cheaper than catching them later in review or CI.



# ENTERPRISE CODE REVIEW PRACTICES

*Code reviews are a team quality gate — they improve design, reduce risk, and spread knowledge across the team.*


**WHAT TO REVIEW**

- Correctness — does it do what it should, handle edge cases?
- Design & architecture — simple, extensible, well-structured?
- Readability, performance, security, and tests.

# CODE REVIEW — REVIEW PRINCIPLES

*Seven principles for reviews that catch real risk without slowing the team down.*

**REVIEW PRINCIPLES**

- Focus on the code, not the author — be constructive.
- Look for real risks, not style; automate style checks.
- Balance speed and quality — don't block progress needlessly.
- Keep pull requests small; explain intent and risk.
- Separate blockers from suggestions; track open comments.
- Review architecture and behavior, not personal preference.
- Require evidence for performance claims.

*Backed by tooling: GitHub/GitLab PR workflows, CI/CD integration, and static analysis as quality gates.*

**KEY TAKEAWAY** Great reviews create great code and great engineers — make code review a positive, consistent habit.

# DEFENSIVE PROGRAMMING AT TRUST BOUNDARIES

*Validate, normalize, and reject invalid data at the boundary — treat all external input as untrusted until proven otherwise.*

### Validate

- Check inputs at the boundary; reject null, blank, or out-of-range values immediately.

### Normalize

- Convert to a consistent internal form (types, casing, trimming) before use.

### Reject Invalid State

- Fail fast on invalid state; avoid ambiguous null returns -- use Optional or a documented exception where absence is exceptional.

### Avoid Exposing Internal Details

- Don't leak internal representations, stack traces, or implementation details across the boundary.

### Prefer Immutable Inputs

- Use final fields and immutable objects so validated state can't change unexpectedly.

### Log Safely

- Log enough context to diagnose issues without recording sensitive data.

# DEFENSIVE PROGRAMMING — ANTI-PATTERNS TO AVOID

*Failure modes that undermine defensive code at the boundary.*

**ANTI-PATTERNS TO AVOID**

- Leaking internal exceptions or stack traces across the boundary; trusting caller-supplied data without checks; logging sensitive data; using Optional indiscriminately for every field, parameter, or return type.

**KEY TAKEAWAY** Defensive code fails loudly at the boundary -- catching bad data there is far cheaper than debugging it in production.

# DEFENSIVE PROGRAMMING IN PRACTICE

*Validate, normalize, and fail fast at the boundary — then keep state immutable.*

**BEFORE / AFTER EXAMPLE**

```
// Bad — trusts the caller, mutable, ambiguous null
class Registration {
    public String email;
    Registration(String email) { this.email = email; }
}
// Good — validated at the boundary, immutable, fails fast
final class Registration {
    private final String email;
    Registration(String email) {
        if (email == null || email.isBlank())
            throw new IllegalArgumentException("email");
        this.email = email.trim().toLowerCase();
    }
}
```

**KEY TAKEAWAY** Fail fast at the boundary, then make invalid state impossible to represent.

# TRY IT YOURSELF — EXERCISE 5: CORRELATION TODOS (STARTER)

*Reinforces: the Log Safely guidance you just covered — a fill-in-the-blank starter, not from scratch.*

| STEP | WHAT TO DO |
|---|---|
| **Goal** | Complete fill-in-the-blank TODOs for correlation one-liners used during refactor notes |
| **Starter** | TODO / fill-in-the-blank lines — replace every ____; not a from-scratch write-up |
| **File** | lab12-correlation-todos.md (in examples/module-12-exercises/notes/) |
| **Fill with** | lab-request-001, short log phrases, and a PII field you must never log (e.g. raw email) |
| **One-liner rule** | Every public service entry logs correlation once |
| **Reference** | exercises/exercise-05-fill-correlation-oneliner-todos.md |

# TRY IT YOURSELF — EXERCISE 5: CORRELATION TODOS (STARTER)

*Reference code — compare your answer, then continue.*

```
$ cat lab12-correlation-todos.md
  Correlation id value: _____
  Log on activate entry: _____
  Log on activate success for Ravi: _____
  Never log field: _____   // PII -- e.g. raw email
```

**TRY IT NOW** Complete Exercise 5 (fill in every blank) before continuing — see exercises/exercise-05-fill-correlation-oneliner-todos.md.

# TRY IT YOURSELF — EXERCISE 6: LAB 12 PREP CHECKLIST

*Capstone: confirm you're ready for the lab — pre-lab prep only, not the full Lab 12 refactor.*

| STEP | WHAT TO DO |
|---|---|
| **Goal** | Confirm standards prep is complete without doing the full refactor lab |
| **File** | lab12-prep-checklist.md (in examples/module-12-exercises/notes/) |
| **Artifacts** | Confirm smell bingo, equals sheet, API sketch, and correlation TODOs all exist |
| **Fixtures** | Recall Amina ACTIVE and Ravi PROSPECT |
| **Pass/Fail** | Pass if the apply/defer SOLID note exists; otherwise revisit Exercise 5 |
| **Reference** | exercises/exercise-06-lab12-prep-checklist.md |

# TRY IT YOURSELF — EXERCISE 6: LAB 12 PREP CHECKLIST

*Reference code — compare your answer, then continue.*

```
$ cat lab12-prep-checklist.md
  [ ] Smell bingo card saved        (Exercise 1)
  [ ] Equals vs == cheat sheet saved  (Exercise 2)
  [ ] Target API sketch + SOLID note saved (Exercises 3, 5)
  [ ] Correlation TODOs filled -- lab-request-001 (Exercise 4)
```

**TRY IT NOW** Complete Exercise 6 before Lab 12 — see exercises/exercise-06-lab12-prep-checklist.md.

# LAB OVERVIEW – CODING STANDARDS AND REFACTORING

*Analyze non-compliant legacy code and refactor it to improve readability, maintainability, and design.*

## SCENARIO & STARTING QUALITY PROBLEMS

- You are maintaining a legacy CRM CustomerService that has grown inconsistent, duplicated, and poorly structured over time -- long methods, duplicated logic, and unclear names make it risky to change.
- Bring it in line with enterprise coding standards and refactor for long-term maintainability.

## WORKFLOW

- Analyze the existing code and inspection results.
- Refactor using proven techniques, in small steps.
- Verify behavior with tests, then document changes.

*CONSTRAINTS  Prerequisites: Core Java, JUnit 5, IntelliJ IDEA, Maven. Time-boxed to the lab session -- deliverables include refactored code, tests, and a refactoring report.*

**KEY TAKEAWAY** Good code is not written once — it is continuously improved. Refactor in small steps, commit often.

# LAB TASKS AND DELIVERABLES

*Apply coding standards and refactoring techniques to improve quality, readability, and maintainability.*

| # | TASK | DETAILS | FORMAT |
|---|---|---|---|
| 1 | Set Up the Project | Clone starter repo; import into IntelliJ IDEA. | n/a |
| 2 | Run Code Analysis | Inspect Code; review smells, complexity, violations. | Report |
| 3 | Review Coding Standards | Compare code against the provided standards doc. | n/a |
| 4 | Refactor the Code | Extract Method/Class, Remove Duplication, etc. | .java |
| 5 | Run & Fix Tests | Ensure all unit tests pass after refactoring. | Test results |
| 6 | Document Changes | Summarize issues found and improvements made. | .md/.docx |

# LAB TASKS — SUCCESS CRITERIA

*How your refactoring submission will be evaluated.*

**SUCCESS CRITERIA**

- All critical/high inspection issues addressed; all unit tests pass; code follows the provided standards; refactoring report clearly summarizes the work.
- Each issue: fixed, suppressed with justification, or documented as not applicable -- don't change code just to satisfy a tool.
- Show behavior preserved: before/after test results, preserved public behaviors, before/after complexity evidence, and a commit per step.
- Estimated effort: 45 minutes; branch as feature/lab-refactor, commit frequently, open a PR before the deadline.

**KEY TAKEAWAY** Refactor incrementally, run tests often, and never change behavior — let tests be your safety net.

# JAVA CODE QUALITY: DEFINITION OF DONE

*A practical checklist for whether code is genuinely ready -- not just compiling, but meeting the module's standards.*

1. Naming communicates intent.
2. Formatting is automated.
3. Methods and classes are cohesive.
4. Dependencies point in the correct direction.
5. Inputs are validated at boundaries.
6. Exceptions preserve useful context.
7. Duplication and unnecessary complexity are minimized.
8. Tests protect behavior.
9. Static-analysis findings are resolved or justified.
10. Code review is complete; documentation explains non-obvious decisions.

*Bottom line: treat this checklist as your Definition of Done for any Java change -- not every box needs a tool, but every box needs an answer.*

**KEY TAKEAWAY** Clean code is not written once -- it's a continuous practice. Use this checklist before you call any change done.

# KNOWLEDGE CHECK (1 OF 2)

*Test your understanding of coding standards, code smells, and refactoring techniques.*

**1. What's the primary goal of coding standards?**

**A.** Adding new features
**B.** To reduce lines of code
**C. Consistency, readability, and maintainability**
**D.** To eliminate all comments

**2. Which is an example of a code smell?**

**A.** Meaningful comments
**B. A class with 5000 lines of code**
**C.** Proper use of constants
**D.** Clear method names

**3. What does refactoring mean?**

**A.** Adding new features
**B. Improving structure without changing behavior**
**C.** Deleting unused code
**D.** Rewriting from scratch

**4. A method has six related address parameters and is called from five locations. Which refactoring fits best?**

A. Extract Method
**B. Introduce Parameter Object**
C. Rename
D. Replace Magic Number

**KEY TAKEAWAY** Refactoring improves internal structure while preserving behavior — always keep tests as your safety net.

# KNOWLEDGE CHECK (2 OF 2)

*Four more questions, plus the full answer key.*

**5. Tests still pass, but exception type and logging changed. Behavior-preserving?**

A. Yes -- passing tests always means behavior is preserved

**B. Not necessarily — exceptions are behavior too**

C. Only if performance also improves

D. No -- refactoring can never change any code

**6. What's a key benefit of code reviews?**

**A.** They replace automated tests

**B. They catch issues the original author may miss**

**C.** They slow down development

**D.** They're only for senior developers

**7. Why should methods be small and focused?**

**A.** They use fewer variables

**B. They're easier to test, understand, reuse**

**C.** They compile faster

**D.** They reduce class count

**8. What should you do BEFORE refactoring code?**

**A.** Remove all comments

**B. Ensure tests exist to verify behavior**

**C.** Add more features

**D.** Change all method names

**ANSWER KEY**

- **1-C 2-B 3-B 4-B 5-B 6-B 7-B 8-B**

**KEY TAKEAWAY** Write code that is easy to read, easy to change, and easy to trust — better code, better quality, better software.

*"Write clean. Review often. Refactor always."*

MODULE 12 COMPLETE

# Java Coding Standards and Best Practices

You can now write clean, consistent, maintainable Java code and apply proven refactoring techniques with confidence.